arXiv:2605.29486v2 [cs.CL] 14 Aug 2026

PHONEWORLD: SCALING PHONE-USE AGENT ENVIRONMENTS

Yuxuan Liu[*1,2], **Xin Lai**[*1], **Junyi Li**[*1], **Pengyuan Lyu**[*1], **Jason**[*1], **Yiduo Guo**[1], **Zhengyao Fang**[1], **Yang Ding**[1], **Yi Zhang**[1], **Weinong Wang**[1], **Huawen Shen**[1], **Xingran Zhou**[1], **Liang Wu**[1], **Fei Tang**[1], **Sunqi Fan**[1], **Shangpin Peng**[1], **Zheng Ruan**[1], **Anran Zhang**[1], **Chengquan Zhang**[†1], **Han Hu**[1], **Benyou Wang**[4], **Ji-Rong Wen**[2], **Rui Yan**[‡3], **and Zhengyang Tang**[‡1,4]

[1]Tencent Hunyuan
[2]Gaoling School of Artificial Intelligence, Renmin University of China
[3]Wuhan University
[4]The Chinese University of Hong Kong, Shenzhen

Code APKs

ABSTRACT

A central bottleneck for phone-use agents is that controllable, reproducible environments covering real mobile behavior are hard to build at scale. Existing mobile-agent benchmarks have made important progress on evaluation, but they do not by themselves provide a scalable way to construct many new phone-use environments. We present PhoneWorld, a reusable pipeline that converts real GUI trajectories and screenshots into controllable phone-use environments, executable tasks, automatic verifiers, and training rollouts. Rather than hand-building one mobile benchmark at a time, PhoneWorld uses real trajectories to recover which screens matter, how screens connect, which interactions must change environment state, and which user goals admit automatic verification. From these signals, it builds runnable mock Android apps backed by read-only app content and mutable state, then derives executable tasks, rule-based verifiers, and training rollouts from the same environments. In its current instantiation, PhoneWorld covers 34 apps across 16 domains, spanning common consumer mobile behaviors such as search, browsing, shopping, booking, media, and social interaction. Under a fixed training budget, replacing 10K steps from an auxiliary AndroidWorld corpus in an AndroidWorld-based baseline with broad PhoneWorld supervision improves all four evaluation benchmarks at once, raising HYMobileBench by 17.7 points, AndroidControl by 6.0 points, AndroidWorld by 14.7 points, and PhoneWorld by 52.5 points. We then study two additional scaling questions: increasing the amount of PhoneWorld supervision strongly improves PhoneWorld performance, and under a fixed PhoneWorld budget, expanding app coverage yields even larger gains. Finally, a downstream reinforcement-learning case study shows that PhoneWorld's automatically verified mock-app rewards transfer beyond PhoneWorld itself: adding PhoneWorld mock-app RL on top of real-app RL improves a 4B phone agent from 40.67% to 45.33% on a 150-task real-phone human evaluation and from 77.2% to 83.2% on AndroidWorld. Overall, PhoneWorld shifts the focus from building one mobile benchmark at a time to scaling verifiable phone-use environments for evaluation, supervision, and reinforcement learning.

∗: Equal contribution.
†: Project Lead.
‡: Corresponding authors.

1 INTRODUCTION

Recent progress in multimodal and computer-use agents has renewed interest in phone-use agents that operate smartphones directly from pixels (Wang et al., 2024; 2025b; Zhang et al., 2025; Qin et al., 2025; Wang et al., 2025a; Zhou et al., 2025; Xu et al., 2026a). Unlike API-based agents, phone-use agents must handle visually rich, stateful, touch-driven interfaces across many mobile apps (Liu et al., 2025; Huang et al., 2025; Liu et al., 2026). Progress in this area is therefore limited not only by model capability, but also by environment supply. Real mobile apps change frequently, are hard to reset, and are expensive to turn into reproducible evaluation and training environments (Xu et al., 2025b; Shi et al., 2025; Luo et al., 2025; Chen et al., 2025; Xi et al., 2026). As a result, scaling phone-use agents also requires scaling phone-use environments.

Existing mobile-agent benchmarks have made important progress on reliable evaluation (Xi et al., 2026). AndroidWorld (Rawles et al., 2025) shows that agents can be evaluated reproducibly in real Android apps, while MobileWorld (Kong et al., 2025) extends evaluation to longer and more complex mobile tasks. A3 introduces Android Agent Arena (Chai et al., 2025), a real-world online benchmark for mobile GUI agents that evaluates agents on dynamic Google Play apps using essential-state-based task verification. These are valuable contributions, but they mainly focus on evaluation after an environment has already been built. Our focus is different. We ask how to build many new phone-use environments in a way that can scale, especially for mainstream consumer-facing apps.

In this paper, we present PhoneWorld, a reusable pipeline that converts real GUI trajectories and screenshots into controllable phone-use environments, executable tasks, automatic verifiers, and training rollouts. We use real trajectories not just as demonstrations to imitate, but as guidance for environment construction. They reveal which screens are central, how navigation flows move between them, which interactions must persist into mutable state, and which user goals can later be checked automatically.

Concretely, PhoneWorld first recovers a prioritized screen inventory, transition graph, and state-changing interactions from real usage traces. It then uses representative screenshots to build a runnable mock Android app backed by read-only app content and mutable state, and derives executable tasks, programmatic verifiers, and successful rollouts from that environment. This keeps the environments grounded in real mobile behavior while making them resettable, inspectable, and reusable. In the current paper, we instantiate this pipeline on 34 apps across 16 domains, spanning common consumer mobile behaviors such as search, browsing, shopping, booking, media consumption, and social interaction.

PhoneWorld is therefore not just another benchmark. It is a reusable way to keep building new phone-use environments and turning them into both evaluation tasks and training data. This matters because the same environment that supports programmatic evaluation can also be reset, re-executed, and harvested for successful rollouts that become supervision for model training.

Empirically, we organize the results around three scaling questions. First, under a matched total training budget, can partially replacing steps from an auxiliary AndroidWorld corpus with broad PhoneWorld supervision improve a strong AndroidWorld-based baseline? Second, how does performance change as the amount of PhoneWorld supervision scales? Third, under a fixed PhoneWorld budget, how does performance change as app coverage scales? The answers are consistent: replacing 10K steps from the auxiliary AndroidWorld corpus with PhoneWorld supervision drawn from 34 apps improves HYMobileBench by 17.7 points, AndroidControl by 6.0 points, AndroidWorld by 14.7 points, and PhoneWorld by 52.5 points. A full-replacement control further shows that PhoneWorld supervision is strong but complementary to AndroidWorld data: replacing the entire auxiliary AndroidWorld corpus strongly improves PhoneWorld performance, but does not yield the strongest all-around matched-budget setting. We further validate PhoneWorld as a reinforcement-learning environment in a downstream 4B phone-agent case study: adding PhoneWorld mock-app RL on top of real-app RL improves real-phone task success from 40.67% to 45.33% and improves AndroidWorld from 77.2% to 83.2%. These results support our central claim: progress in phone-use agents depends not only on stronger models, but also on a scalable way to build verifiable phone-use environments.

Our contributions are as follows:

- We present PhoneWorld, an AI-driven, human-audited pipeline that turns real GUI traces into controllable phone-use environments, executable tasks, automatic verifiers, and training rollouts.
- We instantiate this pipeline on 34 consumer-facing mobile apps across 16 domains, producing runnable environments with executable tasks and rule-based verification.
- We show that PhoneWorld supports both evaluation and training: under a matched training budget, replacing 10K steps from an auxiliary AndroidWorld corpus with broad PhoneWorld supervision improves all four evaluation benchmarks, while a full-replacement control shows that PhoneWorld supervision is strong but complementary to AndroidWorld data.
- We provide scaling studies showing that both more PhoneWorld supervision and broader app coverage improve performance, with app coverage emerging as the strongest scaling signal under fixed PhoneWorld budgets.
- We demonstrate that PhoneWorld is RL-ready: its automatic verifiers can be used as mock-app rewards, and a downstream real+mock RL case study improves both real-phone evaluation and AndroidWorld beyond real-app RL alone.

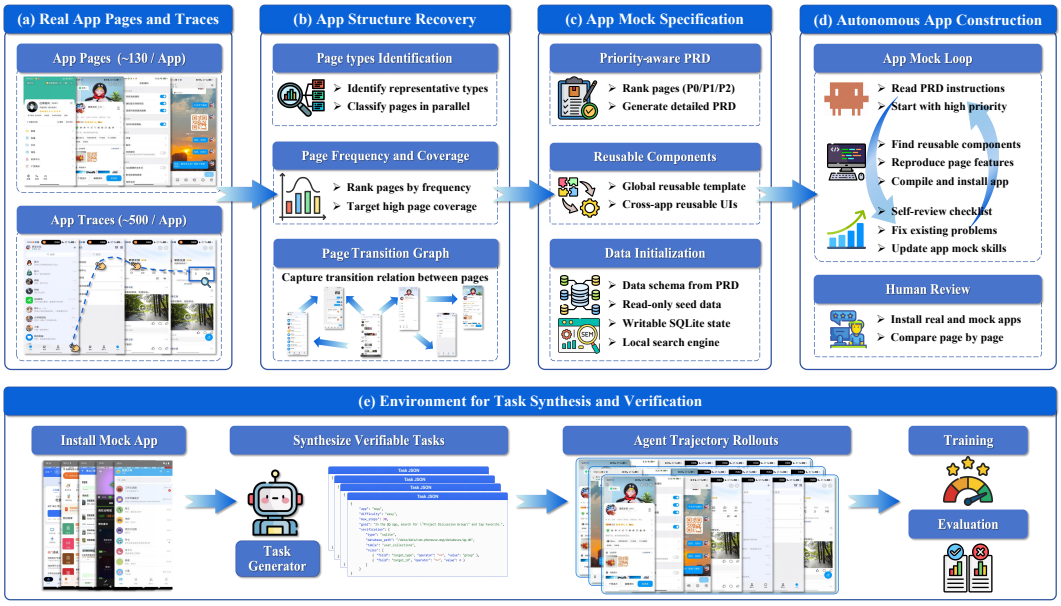


Figure 1: **PhoneWorld turns real GUI traces into runnable phone-use environments. (a) Real-app traces**: representative screenshots and manually collected exploratory-use episodes from the real app. **(b) Structure recovery**: screenshots are classified into page types, then visit frequencies and the page-transition graph are extracted from these trajectories. **(c) Build specification**: the recovered structure is converted into page-level PRDs, reusable UI components, and a read-only content layer plus mutable-state schema. **(d) App construction**: a coding agent iteratively implements, compiles, tests, and revises the mock Android app, followed by human audit. **(e) Tasks, verification, and rollouts**: the runnable environment supports executable task generation, rule-based verification, agent rollouts on an emulator, and downstream use for both benchmarking and SFT.

2 Related Work

The most closely related line of work studies *mobile-agent benchmarks* (Deng et al., 2024; Xu et al., 2025a;c). AndroidWorld (Rawles et al., 2025) established a strong benchmark for online evaluation in real Android apps, with programmatic task initialization, success checking, and reset logic. MobileWorld (Kong et al., 2025) pushes this line further toward longer-horizon, cross-app, and more realistic mobile tasks. MobileBench-OL (Wu et al., 2026a) provides a comprehensive

Chinese benchmark for evaluating mobile agents in real-world environments, emphasizing realistic app interactions and practical task-execution capability. These benchmarks are important because they make rigorous online evaluation possible. However, their main focus is still evaluation in environments that have already been built. PhoneWorld addresses a different question: how to build many new phone-use environments, especially for mainstream consumer-facing apps, in a way that can scale.

A second related line studies *scalable environment construction* for more general agents (Zala et al., 2024; Wang et al., 2026b; Cao et al., 2026; Xu et al., 2026b). InfiniteWeb (Zhang et al., 2026b) automatically generates functional multi-page websites with task-centric specifications and verifiable evaluators. AutoWebWorld (Wu et al., 2026b) models web environments as finite-state machines and translates them into interactive websites. GUI Exploration Lab (Yan et al., 2026) instead constructs a controllable simulation engine for screen-navigation research, using multi-turn reinforcement learning to study exploration and generalization. Agent-World (Dong et al., 2026), for example, studies how to synthesize many tool- and database-based environments for general agent training and evaluation. This is close in spirit to PhoneWorld: both works care about environment scale, controllability, and the link between evaluation and training. The key difference is the interaction setting. Agent-World focuses on general tool-using agents operating over tools, databases, and MCP-style interfaces. PhoneWorld focuses on phone-use agents that must act through pixels, touch interaction, mobile navigation, and app state.

More broadly, our work also connects to a growing view that environments should support not only evaluation but also data generation for training. PhoneWorld follows this view in a phone-use setting. The same pipeline that builds a runnable app also produces executable tasks, automatic checks, and successful rollouts that can be turned into supervised trajectories. In this sense, PhoneWorld is closest to prior benchmark work in its evaluation goals, and closest to scalable environment synthesis work in its overall role. Our contribution is to bring these two ideas together for GUI-based mobile agents.

3 METHOD

PhoneWorld is designed to repeatedly construct phone-use environments rather than handcraft one benchmark at a time. For each target app, the pipeline first recovers what must be built from real GUI trajectories and screenshots, then constructs a runnable mock Android app backed by read-only app content and mutable state, and finally derives executable tasks, automatic checks, and rollout trajectories from the resulting environment. We do not attempt to replicate every detail of the real app. Instead, we preserve the screens, navigation paths, visible content, and state-changing operations that matter most for phone-use agents. Figure 1 gives the overall pipeline, and Figure 2 shows a concrete worked example.

3.1 INPUTS AND DESIGN SCOPE

Our input for each app consists of representative screenshots and a set of real usage episodes. Each episode contains a natural-language user instruction together with a sequence of screenshots and actions recorded on the real device. These episodes are manually collected by human operators interacting with the real app in an ordinary exploratory manner rather than executing a fixed benchmark script. These two sources serve complementary roles. Screenshots reveal the visual layout and content of each page, while trajectories reveal usage structure: which screens are frequently visited, which transitions are common, and which user goals recur. We use both signals jointly to determine what to build, what to simplify, and what to verify.

This distinction is important. If we only copied screenshots, we would mainly recover appearance. If we only imitated trajectories, we would mainly recover demonstrations. PhoneWorld uses the two together to construct environments that are visually grounded in the real app while preserving its functional behavior. In practice, the screenshots tell us what a page should look like, while the trajectories tell us how the app is actually used and which pages and interactions deserve priority. The exploratory-use collection protocol also matters for the later frequency analysis: the visitation statistics are intended to reflect ordinary app usage rather than a narrow scripted path.

3.2 App Structure Recovery

Building a faithful mock environment requires knowing not just what pages exist, but which ones matter most and how they connect. A real consumer app may contain dozens of distinct screens, yet only a subset drives the majority of user interactions. The goal of this stage is to recover that functional skeleton—a prioritized set of page types together with their navigation relationships—so that subsequent construction effort is concentrated where it matters most for phone-use agents.

We first establish a **page taxonomy** for the target app. We prompt Claude Code to browse representative screenshots and identify recurring page types (e.g., home pages, detail pages, and profile pages), producing a per-app taxonomy of 25–30 categories together with a classification prompt that describes each category. A lightweight vision-language model then classifies the full screenshot corpus into this taxonomy in parallel, and the results are aggregated to produce a per-type inventory. Given this classified corpus, we then derive a **page frequency distribution** by mapping each screenshot in the trajectory to its corresponding page type and counting occurrences across all episodes. This distribution directly determines a priority ranking: pages visited most frequently are assigned P0 (must build), moderately visited pages receive P1 (recommended), and long-tail pages are marked P2 (built only if required by downstream tasks). This frequency-driven prioritization ensures that development effort tracks real usage patterns rather than subjective judgments about feature importance. To construct the overall blueprint of the target app, we also extract a **page transition graph** that encodes navigation flows between page types. For each episode, every consecutive pair of pages visited produces a directed edge; aggregating edges across all episodes yields a weighted graph whose high-weight edges identify the dominant navigation paths. These paths inform which inter-page connections the mock environment must preserve to support realistic agent navigation.

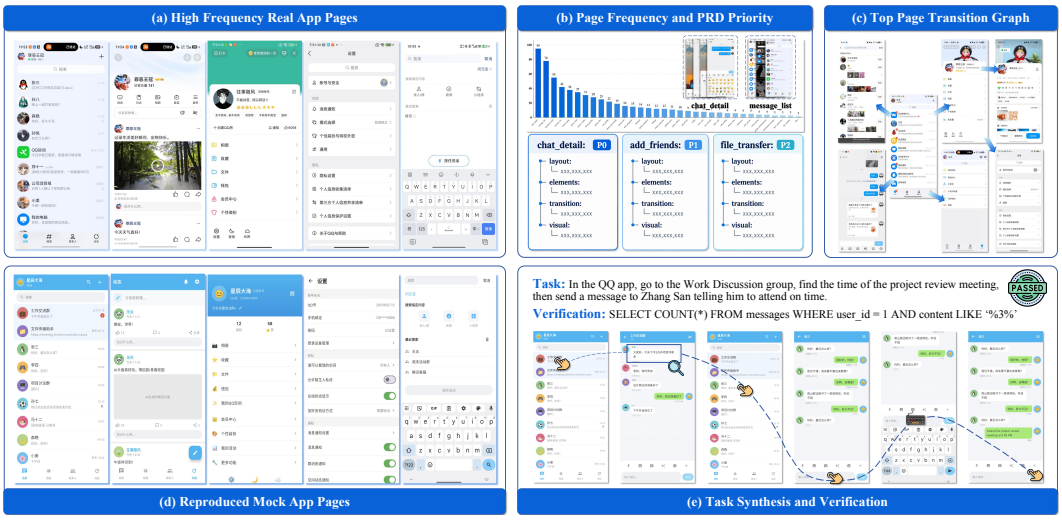


Figure 2: **Worked example of constructing a QQ-like PhoneWorld environment. (a) High-frequency real-app pages**: representative screenshots of the most visited page types identified from deduplicated app screenshots. **(b) Page frequency and build priorities**: trajectory statistics determine page priorities (P0/P1/P2), and each page receives a structured PRD. **(c) Transition graph**: dominant navigation flows are extracted from sequential page visits in the trajectories. **(d) Generated mock pages**: corresponding screens are implemented from the PRD and reusable component library. **(e) Task and verifier**: a representative task requiring information seeking and messaging, checked automatically with a SQLite query.

3.3 Build Specification Generation

The prioritized page list and transition graph tell the coding agent *what* to build, but not *how*. This stage turns the recovered structure into a concrete build specification: per-page feature requirements, a shared library of reusable components, and a data architecture that supports each app.

For each page, we generate a **structured PRD** (product requirements document) ranked by priority. A vision-language model examines two or three representative screenshots of the page type and produces a specification along four dimensions: page layout, interactive elements, transition relations, and visual attributes. These per-page PRD entries become the primary instruction set for the construction agent—they tell it not just which pages to build, but exactly what should appear on each one. Many pages share interaction components across apps (e.g., search bars, personal profiles). We consolidate these into a **reusable component library**. When the coding agent encounters such a pattern in a PRD, it instantiates the corresponding module rather than reimplementing it, so per-app effort is concentrated on what is genuinely app-specific. To support each app at runtime, we design a **data architecture** that separates read-only app content from mutable app state. The read-only app content stores the entities, records, and page content exposed at initialization, allowing agents to browse, search, and query realistic data without network access. The mutable app state is stored in a resettable SQLite database whose tables are updated deterministically by user actions such as favoriting an item, modifying a cart, posting a comment, or sending a message. Downstream verifiers query the same database after rollout completion, and environment reset restores the initial content snapshot so tasks can be re-executed repeatedly from a known state. Because these environments run offline, we also build a local BM25 search engine over the read-only app content, giving agents deterministic retrieval behavior across runs. This database-backed state model is also what makes verifier-confirmed trajectory harvesting and future online training practical.

3.4 Autonomous App Construction

The coding agent constructs each app in an iterative loop: it reads the PRD, generates Kotlin/Jetpack Compose source code, compiles the APK, runs a self-review checklist, and fixes reported issues. In practice, a single app typically takes several iterations to resolve navigation interdependencies, data loading, and UI rendering, ultimately reproducing the target app's required screens, interactions, and functionality.

Building different apps creates a natural feedback loop that improves the process over time. First, each compilation failure or runtime bug encountered during self-review is diagnosed and catalogued. We accumulate these into a structured checklist—currently covering issues such as schema mismatches, dead buttons, and missing routes—that the agent runs after every build pass. Second, when the same UI or logic pattern appears across multiple apps (e.g., search with offline retrieval, comment lists), we extract it into a reusable component that subsequent apps can instantiate directly from their PRD. The current library contains 18 such modules. Third, design experience from earlier apps is distilled into construction skills that the agent follows in later builds, making subsequent apps progressively cheaper and more reliable to construct.

3.5 Human-in-the-Loop Quality Assurance

PhoneWorld is AI-driven but human-audited. After autonomous construction, we install the compiled app on an emulator and run smoke tests covering core flows: app launch, tab switching, search, detail-page navigation, and representative write operations. These automatic checks catch many common issues before human reviewers are involved. Human review focuses on high-impact issues that are difficult to reduce to static rules. Reviewers compare the mock app against the corresponding real app side by side and report differences in layout, navigation, data coverage, visual realism, or stateful behavior. The key requirement is not pixel-perfect matching, but whether the main user flows function correctly and the interface is close enough to the real app for a phone-use agent to operate on it. Reported issues are fed back into the construction loop, after which the app is rebuilt and retested. This review cycle typically converges within one to two rounds. The combination of AI-driven construction with targeted human auditing allows the system to scale without claiming unrealistic full automation.

3.6 Task Synthesis and Verification

Controllable environments are especially valuable for agent training because they enable scalable task generation and rule-based verification, which are both difficult to obtain from real apps at scale (Zhang et al., 2026a; Lu et al., 2025; Wang et al., 2026a). Writing tasks by hand does not scale: each new app would require a human to invent goals, trace verification paths, and confirm

consistency with the underlying data. PhoneWorld avoids this bottleneck by generating tasks directly from the artifacts already produced during app construction—the read-only app content, the database schema, and the UI specification. This ensures that every generated task references entities the agent can actually see and triggers state changes the verifier can actually check.

Example of Synthesized Task and Verification Rule

```
{
  "app": "mqq",
  "difficulty": "easy",
  "max_steps": 10,
  "goal": "In QQ, search for \"Project Group\" and tap Favorite.",
  "verification": {
    "type": "sqlite",
    "database_path": "/data/data/com.phoneuse.mqq/databases/qq.db",
    "table": "user_collections",
    "rules": [
      { "field": "target_type", "operator": "==", "value": "group" },
      { "field": "target_id", "operator": "==", "value": 4 }
    ]
  }
}
```

Figure 3: Example of a synthesized verifiable task. Each task is represented as a structured JSON object. The verification field specifies a database-level rule that enables automatic task-completion checking without manual annotation.

For each task, the generator produces a **grounded task specification**. To ensure that every task is consistent with what the environment actually supports, the generator cross-references three sources: read-only app content (what content exists), database schema (what state changes are possible), and per-page PRD (what is visible on screen). This grounding guarantees that generated goals reference real entities, request achievable operations, and admit deterministic verification—allowing the task pool to scale with less human annotation effort. We employ two styles of **rule-based verification**. For information-seeking tasks, we check whether the agent's final answer contains key values drawn from the read-only app content. For state-changing tasks, we query the SQLite database directly and verify that the expected records exist. Both styles are deterministic, require no model-based judging, and eliminate evaluator variance across runs. We also generate **cross-app tasks** by identifying shared entities across two apps' read-only app content and producing goals that require retrieving information from one app and acting on it in another.

The controllability of these environments makes them useful beyond one-time evaluation. Because each app can be reset to a known state, tasks can be re-executed indefinitely, and outcomes can be verified deterministically, the same infrastructure supports three complementary uses: scaling up supervision by generating rollouts across the full task pool and retaining only verified successes, providing precise evaluation through rule-based checks that eliminate model-based judging, and enabling controlled training where the data composition, app coverage, and difficulty distribution can all be varied systematically. This is the core advantage of building environments rather than collecting static datasets—the environments remain a live, reusable source of both evaluation and training signal.

From verifiers to RL rewards. The same verifier design also makes PhoneWorld directly usable for reinforcement learning. Each task verifier can be interpreted as a binary completion reward after a rollout: answer-based tasks check whether the final response contains the required grounded value, and state-changing tasks check whether the reconstructed app state contains the expected update. Since the environment can be reset and the verifier is programmatic, reward computation does not require model-based judging or manual human review. This property distinguishes PhoneWorld from a static trajectory dataset: it can serve as an online, repeatable, automatically checked mock-app RL environment.

3.7 THE PHONEWORLD SUITE

The preceding sections describe how PhoneWorld constructs individual environments. However, the practical value of the system depends not only on the quality of each app, but on the breadth, reusability, and dual-use nature of the resulting suite. This section summarizes what the current PhoneWorld suite contains and how it supports both evaluation and training from the same infrastructure.

The suite provides broad **app coverage**: 34 mock Android apps spanning 16 consumer-facing domains—short video, social media, shopping, food delivery, travel, music, reading, finance, and others. These apps collectively exercise the most common phone-use behaviors: search, feed browsing, detail-page navigation, content interaction, publishing, and ordering, all through runnable interfaces backed by read-only app content, mutable state, and programmatic checks. Importantly, the 34 apps are not built independently from scratch. They share a library of 18 **reusable modules**—search engine, feed cards, comments, cart and checkout, address manager, messaging, settings, media player, among others—so that adding a new app reuses proven building blocks and concentrates development effort on genuinely app-specific logic.

For evaluation, we maintain an **audited benchmark** of 120 tasks: 102 single-app tasks (three per app) and 18 cross-app tasks spanning easy, medium, and hard difficulty levels. Each task is paired with an automatic verifier—either answer-based or SQLite-based—and is manually reviewed to ensure correctness. The benchmark is intentionally compact so that it can serve as a stable, low-noise evaluation target. These 120 tasks are strictly held out from the generated task pool used for training; training and evaluation share the same set of apps but draw from disjoint task instances.

For training, the same suite extends well beyond the audited benchmark. We maintain a **generated task pool** of 7,936 tasks produced by the task synthesis stage described above. Executing agents on this pool and retaining only verifier-confirmed successes yields 3,354 successful episodes totaling 36,193 interaction steps, which form the PhoneWorld training corpus used in our later experiments. Because environments, tasks, verifiers, and training episodes all originate from the same controllable app instances, adding a new app to PhoneWorld automatically enlarges both the evaluation surface and the training supply—no separate data-collection campaign is required. Table 1 summarizes the suite.

Artifact provenance and release format. For clarity, the APK artifacts in this suite are RUC Gaoling academic research resources: they were independently developed and maintained by authors affiliated with the Gaoling School of Artificial Intelligence, Renmin University of China, using RUC research resources. Tencent-affiliated collaborators used the RUC-provided environments only as research environments for agent rollout, training, and evaluation. The APK artifacts are not Tencent products, commercial services, or app-distribution artifacts. The external research release uses distinct package names, synthetic seed data, offline emulator-only execution, and "Mock" launcher labels where appropriate; app and brand references identify the interaction patterns being studied rather than any affiliation or endorsement.

Table 1: **Summary of the PhoneWorld suite.**

Component	Summary
Apps	34 mock Android apps across 16 domains
Reusable modules	18 shared modules reused across app families
Benchmark	120 audited tasks: 102 single-app + 18 cross-app
Verification	Answer-based checks and SQLite-based state checks
Task pool	7,936 generated tasks used for large-scale rollout generation
Training rollouts	3,354 successful episodes / 36,193 interaction steps

4 EXPERIMENTAL SETUP

4.1 SCALING QUESTIONS

We organize the experiments around three scaling questions. First, under a matched total training budget, can partially replacing steps from an auxiliary AndroidWorld corpus with broad PhoneWorld supervision improve a strong AndroidWorld-based baseline? We accompany this matched-budget comparison with a full-replacement control that tests whether PhoneWorld supervision is complementary to, rather than interchangeable with, that auxiliary corpus. Second, how does performance change as the amount of PhoneWorld supervision scales? Third, under a fixed PhoneWorld budget, how does performance change as app coverage scales?

4.2 TRAINING PROTOCOL

Our main experiments focus on supervised fine-tuning of the same vision-language backbone, Qwen3.5-9B, using the same training setup and input format across all conditions. All data are converted into a unified multimodal SFT format in which the model receives a system prompt, the current screenshot, the user instruction, and a textual summary of previous actions, and predicts the next thought-and-action output. Training is run in LlamaFactory for two epochs with the same hyperparameters across compared checkpoints. We keep the original mobile screenshot resolution (1080×2400), use normalized coordinates, and avoid changing the image preprocessing between training and inference.

4.3 TRAINING CORPORA AND COMPARED SETTINGS

We construct three trajectory corpora. The first is a shared AndroidWorld base corpus comprising 36,193 interaction steps from Gemini 3.1 Pro rollouts on AndroidWorld tasks. The second is an auxiliary AndroidWorld corpus, also 36,193 interaction steps, collected from Seed 2.0 Pro rollouts on the same benchmark. The third is a PhoneWorld rollout corpus collected by running Seed 2.0 Pro on the generated PhoneWorld task pool and retaining only verifier-confirmed successful rollouts, yielding 3,354 trajectories totaling 36,193 interaction steps.

Unless otherwise stated, matched-budget comparisons keep the shared AndroidWorld base corpus fixed and vary only the composition of the remaining 36,193 training steps. The **Baseline** model combines the shared AndroidWorld base corpus with the full 36,193-step auxiliary AndroidWorld corpus. The **10K PhoneWorld replacement** model keeps the same 72,386-step total budget, retains 26,193 auxiliary AndroidWorld steps, and replaces the remaining 10,000 with PhoneWorld successful-rollout steps drawn from 34 apps. The **Full PhoneWorld replacement** model keeps the shared AndroidWorld base corpus but replaces the entire 36,193-step auxiliary AndroidWorld corpus with 36,193 PhoneWorld steps. For the supervision-scaling analysis, we start from the shared AndroidWorld base corpus alone and add 0, 10K, 20K, or 36,193 PhoneWorld steps. For the app-coverage analysis, we fix the PhoneWorld budget at 10K steps and vary only how many source apps those 10K steps are drawn from.

4.4 EVALUATION BENCHMARKS

We evaluate all models on four benchmarks that together cover offline transfer, real-app online transfer, and in-domain PhoneWorld performance. Table 2 summarizes them.

Table 2: **Evaluation benchmarks used in this paper.**

Benchmark	Environment	Metric	Role in the paper
HYMobileBench	Offline	Step SR	Real-device mobile performance proxy
AndroidControl	Offline	Step SR	Android control transfer
AndroidWorld	Online real app	Task SR	Out-of-domain real-app transfer
PhoneWorld	Online mock apps	Task SR	In-domain PhoneWorld evaluation

HYMobileBench is an internal Hunyuan benchmark manually curated on real devices and serves as an offline proxy for real-device phone-use performance. AndroidControl (Li et al., 2024) serves

as a second offline transfer benchmark for Android control. AndroidWorld (Rawles et al., 2025) measures transfer to real Android apps outside the PhoneWorld ecosystem. PhoneWorld measures in-domain performance on the held-out audited PhoneWorld task set. Following each benchmark's standard harness, we report step success rate (step SR) on HYMobileBench and AndroidControl, and task success rate (task SR) on AndroidWorld and PhoneWorld.

4.5 Implementation details

All offline evaluations are run in the same serving environment. AndroidWorld evaluations use the same online evaluation protocol across compared checkpoints. PhoneWorld is evaluated on Android 13 Pixel 6 emulators using the same runner, app release, and model-serving setup for all compared models. In the current setup, PhoneWorld online evaluation is run on six emulators with three vLLM instances. Unless otherwise stated, we keep the model architecture, prompt format, screenshot resolution, and evaluation harness fixed throughout and vary only the training data composition required by each scaling question.

5 Results

Table 3: **Matched-budget partial replacement.** Replacing 10K steps from the auxiliary AndroidWorld corpus with PhoneWorld supervision improves all four benchmarks.

Benchmark	Baseline	10K PhoneWorld replacement	Absolute Change
HYMobileBench	15.5	33.2	+17.7
AndroidControl	53.7	59.7	+6.0
AndroidWorld	56.9	71.6	+14.7
PhoneWorld	12.5	65.0	+52.5

5.1 Partial replacement under a matched training budget

We begin with the first scaling question: under a matched total training budget, can partially replacing steps from the auxiliary AndroidWorld corpus with broad PhoneWorld supervision improve a strong AndroidWorld-based baseline? Both compared models use the same Qwen3.5-9B backbone, the same training setup, and the same 72,386-step total budget. Both also share the same 36,193-step AndroidWorld base corpus. The difference lies in the remaining 36,193 steps. **Baseline** uses the full 36,193-step auxiliary AndroidWorld corpus. **10K PhoneWorld replacement** keeps 26,193 auxiliary AndroidWorld steps and replaces the remaining 10,000 with PhoneWorld supervision drawn from 34 apps. This is the broadest 10K replacement setting in the app-coverage analysis below.

Table 3 shows that replacing a small slice of the auxiliary AndroidWorld corpus with broad PhoneWorld supervision improves all four benchmarks at once. Compared with Baseline, 10K PhoneWorld replacement gains 17.7 points on HYMobileBench, 6.0 points on AndroidControl, 14.7 points on AndroidWorld, and 52.5 points on PhoneWorld. This is the clearest single result in the paper. **The gain is not limited to PhoneWorld; it also transfers to the real-app AndroidWorld benchmark and the two offline benchmarks.**

This result is important for two reasons. First, it shows that PhoneWorld can improve the strongest all-around matched-budget setting rather than only boosting PhoneWorld performance. Second, the PhoneWorld budget here is only 10K steps, so the improvement cannot be explained by simply adding more data. What changes is the breadth of the training environments under a fixed budget. This motivates the broader app-coverage study later in the section.

5.2 Full replacement as a control

We next extend the same replacement axis to the 100% replacement endpoint. This comparison asks whether PhoneWorld supervision is merely stronger than the auxiliary AndroidWorld corpus or whether the two sources are complementary. To isolate this, we fully remove the auxiliary AndroidWorld corpus while keeping the same shared AndroidWorld base corpus and the same total

budget. **Full PhoneWorld replacement** swaps the full 36,193-step auxiliary AndroidWorld corpus for 36,193 PhoneWorld steps.

Table 4: **Matched-budget full-replacement control.** Full replacement strongly improves PhoneWorld while showing that PhoneWorld and AndroidWorld supervision are complementary.

Benchmark	Baseline	Full PhoneWorld replacement	Absolute Change
HYMobileBench	15.5	33.2	+17.7
AndroidControl	53.7	59.3	+5.6
AndroidWorld	56.9	46.6	-10.3
PhoneWorld	12.5	73.3	+60.8

Table 4 shows that PhoneWorld supervision is powerful on its own. Replacing the full auxiliary AndroidWorld corpus with PhoneWorld data yields large gains on PhoneWorld (+60.8), HYMobileBench (+17.7), and AndroidControl (+5.6). This is the most direct evidence that PhoneWorld environments are not only suitable for evaluation but also useful as a source of training data.

At the same time, AndroidWorld falls by 10.3 points. We do not treat this as a contradiction of the main claim. Instead, it shows that the two data sources are not interchangeable. Appendix A provides a supplementary add-only analysis using the same runs as the supervision-scaling study: when PhoneWorld supervision is added on top of the shared AndroidWorld base corpus without removing existing AndroidWorld data, PhoneWorld rises sharply while AndroidWorld remains roughly unchanged (Table 6). This pattern is more consistent with a missing auxiliary AndroidWorld transfer signal than with a harmful effect of adding PhoneWorld supervision itself. PhoneWorld provides strong supervision for mainstream consumer phone-use behaviors and for the PhoneWorld ecosystem itself, while the auxiliary AndroidWorld corpus still contributes a distinct real-app transfer signal. Together with Table 3, this gives the main matched-budget story: **PhoneWorld supervision is strong, but the strongest all-around setting uses partial replacement rather than full replacement.**

5.3 Scaling the amount of PhoneWorld supervision

We now turn to the second scaling question: how does performance change as the amount of PhoneWorld supervision scales? To isolate this effect, we keep the 36,193-step AndroidWorld base corpus fixed and add 0, 10K, 20K, or 36,193 PhoneWorld steps on top of it. Unlike the matched-budget comparison above, this experiment does not include the auxiliary AndroidWorld corpus. The goal is not to maximize overall benchmark performance, but to measure the value of increasing PhoneWorld supervision itself. Figure 4(a) summarizes this scaling curve.

Figure 4(a) shows a clear monotonic pattern. PhoneWorld task success rises from 14.2 to 64.2, 70.0, and 73.3 as more PhoneWorld supervision is added. The largest gain appears in the first 10K PhoneWorld steps, after which the returns become smaller but remain positive. This result shows that scaling the amount of PhoneWorld supervision mainly strengthens PhoneWorld performance.

5.4 Scaling app coverage under a fixed PhoneWorld budget

We now turn to the third scaling question: under a fixed PhoneWorld budget, how does performance change as app coverage scales? To answer this, we fix the training setup and the PhoneWorld budget at 10K steps, then vary how many apps those 10K steps are drawn from. The 5-app, 10-app, 20-app, and 34-app settings use the same PhoneWorld budget; only the breadth of environment coverage changes. The 34-app setting is exactly the 10K PhoneWorld replacement setting from Table 3. Figure 4(b) summarizes this result.

Figure 4(b) shows that **broader app coverage is the strongest scaling signal in the paper**. Under the same 10K PhoneWorld budget, expanding the source app set from 5 to 34 raises PhoneWorld from 46.7 to 65.0 (+18.3), HYMobileBench from 14.9 to 33.2 (+18.3), and AndroidWorld from 61.2 to 71.6 (+10.4), while AndroidControl remains roughly stable overall. Because the PhoneWorld budget is fixed, these gains cannot be explained by simply adding more PhoneWorld data. What changes is the breadth of environments that the model is exposed to.

This result is especially important for the paper's main claim. The previous scaling analysis shows that more PhoneWorld supervision helps. The app-breadth analysis shows something stronger: *what* that supervision covers matters even more. The value of PhoneWorld does not come only from adding more trajectories. It comes from broadening the set of mobile environments that the model is exposed to. This is exactly the sense in which PhoneWorld scales phone-use environments rather than simply scaling one more dataset.

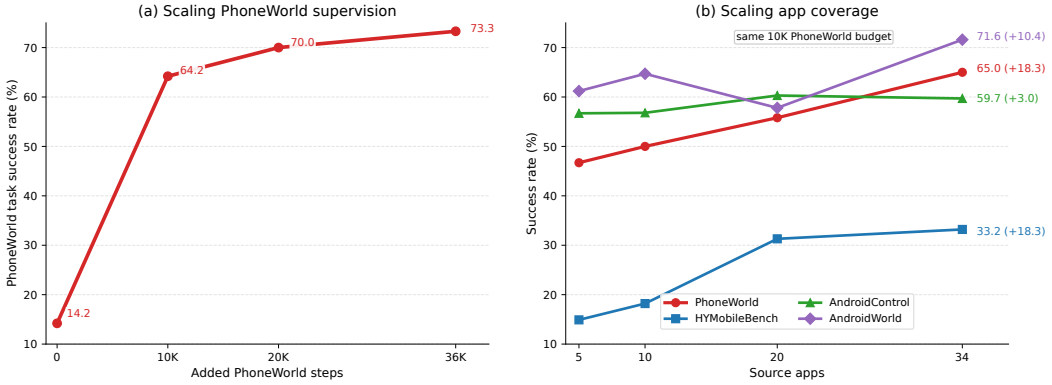


Figure 4: **Main scaling results for PhoneWorld.** Left: scaling the amount of PhoneWorld supervision on top of the shared AndroidWorld base corpus produces a strong monotonic increase in PhoneWorld task success rate. Right: under the same 10K PhoneWorld budget, broader app coverage yields broader and more consistent gains across benchmarks; endpoint annotations show absolute performance together with gains over the 5-app setting.

5.5 Downstream RL case study: PhoneWorld rewards transfer to AndroidWorld

The experiments above use PhoneWorld mainly as a source of verified rollouts for supervised training. We next ask whether the same environments can serve as a reinforcement-learning reward source. This case study uses a separate 4B phone-agent model line, PhoneBuddy (**?**), to isolate the effect of the final RL environment. All compared checkpoints share the same backbone, SFT initialization, action interface, and evaluation protocol; they differ only in whether the final RL stage uses no RL, real-app RL, or mixed real-app plus PhoneWorld mock-app RL. We treat this as downstream evidence for PhoneWorld rather than as a new benchmark: the key question is whether verifier-backed mock-app rewards add transfer value beyond real-app RL alone.

Table 5: **Downstream RL case study.** PhoneWorld mock-app RL improves over real-app RL alone and transfers to AndroidWorld. The real-phone evaluation contains 150 human-judged tasks, with 50 tasks in each category. All values are task success rates (%).

Training recipe	Single-App	Cross-App	Mini-App	Real-phone avg.	AndroidWorld
SFT	34.0	22.0	54.0	36.67	60.3
Real-app RL	54.0	20.0	48.0	40.67	77.2
Real-app + PhoneWorld mock-app RL	**62.0**	18.0	**56.0**	**45.33**	**83.2**

Table 5 shows that PhoneWorld is useful not only for producing offline supervision, but also as an online mock-app RL environment. Compared with real-app RL alone, adding PhoneWorld mock-app RL improves the real-phone average from 40.67% to 45.33% and improves AndroidWorld from 77.2% to 83.2%. The AndroidWorld gain is especially important because AndroidWorld is an external real-app benchmark rather than a PhoneWorld in-domain benchmark. This suggests that PhoneWorld's resettable and automatically verified rewards teach phone-use behaviors that transfer beyond the mock apps themselves. The gains concentrate on single-app and mini-app workflows, while cross-app tasks remain weak; this boundary points to the need for future PhoneWorld versions that explicitly scale cross-app information handoff and persistent multi-app state.

6 Discussion and Limitations

Across the matched-budget partial-replacement result, the full-replacement control, the two scaling analyses, and the downstream RL case study, five findings define the paper's empirical story. First, under a fixed training budget, partially replacing steps from the auxiliary AndroidWorld corpus with broad PhoneWorld supervision improves all four benchmarks at once. Second, full replacement shows that PhoneWorld supervision is strong on its own but complementary to the auxiliary AndroidWorld corpus rather than interchangeable with it. Third, scaling the amount of PhoneWorld supervision mainly strengthens PhoneWorld performance. Fourth, under a fixed PhoneWorld budget, scaling app coverage yields the broadest gains and is the strongest scaling signal in the paper. Fifth, PhoneWorld can serve as an automatically verified mock-app RL environment: when added on top of real-app RL, it improves both real-phone evaluation and AndroidWorld. Together, these results support our central claim that progress depends not only on more data, but on scaling the supply and diversity of verifiable phone-use environments.

The results also clarify the relationship between PhoneWorld and AndroidWorld. PhoneWorld is not a replacement for real-app evaluation. Instead, the two environments capture different parts of the problem. PhoneWorld contributes strong supervision for mainstream consumer phone-use behaviors and provides controllable infrastructure for both benchmarking and training. AndroidWorld still supplies a distinct real-app transfer signal, so the strongest matched-budget SFT setting retains most of the auxiliary AndroidWorld corpus and replaces only a small slice with broad PhoneWorld supervision. At the same time, the downstream RL case study shows that PhoneWorld's automatically checked mock-app rewards can improve AndroidWorld performance when combined with real-app RL, indicating transfer rather than mere in-domain overfitting.

PhoneWorld also has several limitations. The generated apps are selective abstractions of real apps rather than full replicas. We preserve the screens, state changes, and interaction paths that matter most for phone-use agents, but we do not aim for complete feature coverage or perfect system fidelity. Our current benchmark is intentionally compact and manually audited, which improves stability but does not exhaust the behavior space of the full PhoneWorld suite. The downstream RL case study further shows that the current PhoneWorld task pool transfers best to single-app and structurally stable mini-app workflows, while cross-app workflows remain difficult. Future versions should therefore scale multi-app information handoff, artifact transfer, and persistent cross-app state dependencies more explicitly. HYMobileBench is an internal benchmark rather than a public one.

7 Conclusion

We presented PhoneWorld, a reusable pipeline for building controllable phone-use environments from real GUI trajectories and screenshots. Rather than handcrafting one benchmark at a time, PhoneWorld turns real mobile usage data into runnable phone-use environments and the executable tasks, verification rules, and successful rollouts needed to use them for both evaluation and training. This makes PhoneWorld useful not only as a benchmark, but also as infrastructure for generating supervision for phone-use agents.

Our experiments support five main conclusions. Under a matched training budget, partially replacing steps from the auxiliary AndroidWorld corpus with broad PhoneWorld supervision improves all four benchmarks at once. Full replacement shows that PhoneWorld supervision is strong on its own but works best in combination with the auxiliary AndroidWorld corpus. Scaling the amount of PhoneWorld supervision mainly strengthens PhoneWorld performance. Scaling app coverage under a fixed PhoneWorld budget yields the broadest gains and emerges as the strongest scaling signal. Finally, PhoneWorld's verifier-backed mock-app rewards can improve a downstream phone agent beyond real-app RL alone and transfer to AndroidWorld. Taken together, these results suggest that scaling phone-use agents requires scaling the supply, breadth, and verifiability of phone-use environments themselves.

Availability

We plan to release the PhoneWorld code, benchmark tasks, runner, and documentation at `https://github.com/EthanLeo-LYX/PhoneWorld`. The mocked apks are con-

structed by Yuxuan Liu using personal research resource and will be distributed through a gated Hugging Face dataset at https://huggingface.co/datasets/EthanLeoLYX/PhoneWorld-APKsfor non-commercial academic research. Each user must accept the dataset terms independently rather than receiving redistributed APK mirrors. The gated APK release uses distinct package names, synthetic data, offline emulator-only execution, and "Mock" launcher labels where appropriate to make clear that the apps are research mock environments rather than real consumer apps. The downstream PhoneBuddy project is available at https://phonebuddyai.github.io/.

REFERENCES

Yuan Cao, Dezhi Ran, Mengzhou Wu, Yuzhe Guo, Xin Chen, Ang Li, Gang Cao, Gong Zhi, Hao Yu, Linyi Li, et al. Gui-genesis: Automated synthesis of efficient environments with verifiable rewards for gui agent post-training. *arXiv preprint arXiv:2602.14093*, 2026.

Yuxiang Chai, Shunye Tang, Han Xiao, Weifeng Lin, Liang Liu, Hanhao Li, Jiayu Zhang, Pengxiang Zhao, Guangyi Liu, Rongduo Han, et al. A3: Android agent arena for mobile gui agents. 2025.

Yuhan Chen, Yuxuan Liu, Long Zhang, Pengzhi Gao, Jian Luan, and Wei Liu. Step: Success-rate-aware trajectory-efficient policy optimization. *arXiv preprint arXiv:2511.13091*, 2025.

Shihan Deng, Weikai Xu, Hongda Sun, Wei Liu, Tao Tan, Liujianfeng Liujianfeng, Ang Li, Jian Luan, Bin Wang, Rui Yan, et al. Mobile-bench: An evaluation benchmark for llm-based mobile agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8813–8831, 2024.

Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.

Kun Huang, Weikai Xu, Yuxuan Liu, Quandong Wang, Pengzhi Gao, Wei Liu, Jian Luan, Bin Wang, and Bo An. Mobileipl: Enhancing mobile agents thinking process via iterative preference learning. *arXiv preprint arXiv:2505.12299*, 2025.

Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, Zhidan Liu, Steven Hoi, and Yue Wang. Mobile-world: Benchmarking autonomous mobile agents in agent-user interactive, and mcp-augmented environments. *arXiv preprint arXiv:2512.19432*, 2025.

Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37:92130–92154, 2024.

Yuxuan Liu, Hongda Sun, Wei Liu, Jian Luan, Bo Du, and Rui Yan. Mobilesteward: Integrating multiple app-oriented agents with self-evolution to automate cross-app instructions. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pp. 883–893, 2025.

Yuxuan Liu, Weikai Xu, Kun Huang, Changyu Chen, Jiankun Zhao, Pengzhi Gao, Wei Liu, Jian Luan, Shuo Shang, Bo Du, et al. Come: Empowering channel-of-mobile-experts with informative hybrid-capabilities reasoning. *arXiv preprint arXiv:2602.24142*, 2026.

Dunjie Lu, Yiheng Xu, Junli Wang, Haoyuan Wu, Xinyuan Wang, Zekun Wang, Junlin Yang, Hongjin Su, Jixuan Chen, Junda Chen, et al. Videoagenttrek: Computer use pretraining from unlabeled videos. *arXiv preprint arXiv:2510.19488*, 2025.

Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2025.

Yucheng Shi, Wenhao Yu, Zaitang Li, Yonglin Wang, Hongming Zhang, Ninghao Liu, Haitao Mi, and Dong Yu. Mobilegui-rl: Advancing mobile gui agent through reinforcement learning in online environment. *arXiv preprint arXiv:2507.05720*, 2025.

Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.

Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024.

Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Wu, et al. Opencua: Open foundations for computer-use agents. *Advances in Neural Information Processing Systems*, 38:139756–139806, 2026a.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026b.

Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. Mobile-agent-e: Self-evolving mobile assistant for complex tasks. *arXiv preprint arXiv:2501.11733*, 2025b.

Qinzhuo Wu, Zhizhuo Yang, Hanhao Li, Pengzhi Gao, Wei Liu, and Jian Luan. Mobilebench-ol: A comprehensive chinese benchmark for evaluating mobile gui agents in real-world environment. *arXiv preprint arXiv:2601.20335*, 2026a.

Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, Jiayi Zhang, Man Chen, Yenchi Tseng, Zhaoyang Yu, et al. Autowebworld: Synthesizing infinite verifiable web environments via finite state machines. *arXiv preprint arXiv:2602.14296*, 2026b.

Ziqiao Xi, Shuang Liang, Qi Liu, Jiaqing Zhang, Letian Peng, Fang Nan, Meshal Nayim, Tianhui Zhang, Rishika Mundada, Lianhui Qin, et al. Toolgym: an open-world tool-using environment for scalable agent testing and data curation. *arXiv preprint arXiv:2601.06328*, 2026.

Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*, 2026a.

Weikai Xu, Zhizheng Jiang, Yuxuan Liu, Pengzhi Gao, Wei Liu, Jian Luan, Yuanchun Li, Yunxin Liu, Bin Wang, and Bo An. Mobile-bench-v2: A more realistic and comprehensive benchmark for vlm-based mobile agents. *arXiv preprint arXiv:2505.11891*, 2025a.

Weikai Xu, Kun Huang, Yunren Feng, Jiaxing Li, Yuhan Chen, Yuxuan Liu, Zhizheng Jiang, Heng Qu, Pengzhi Gao, Wei Liu, et al. How mobile world model guides gui agents? *arXiv preprint arXiv:2605.10347*, 2026b.

Yifan Xu, Xiao Liu, Xinghan Liu, Jiaqi Fu, Hanchen Zhang, Bohao Jing, Shudan Zhang, Yuting Wang, Wenyi Zhao, and Yuxiao Dong. Mobilerl: Online agentic reinforcement learning for mobile gui agents. *arXiv preprint arXiv:2509.18119*, 2025b.

Yifan Xu, Xiao Liu, Xueqiao Sun, Siyi Cheng, Hao Yu, Hanyu Lai, Shudan Zhang, Dan Zhang, Jie Tang, and Yuxiao Dong. Androidlab: Training and systematic benchmarking of android autonomous agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2144–2166, 2025c.

Haolong Yan, Yeqing Shen, Xin Huang, Jia Wang, Kaijun Tan, Zhixuan Liang, Hongxin Li, Zheng Ge, Osamu Yoshie, Si Li, et al. Gui exploration lab: Enhancing screen navigation in agents via multi-turn reinforcement learning. *Advances in Neural Information Processing Systems*, 38: 10970–11002, 2026.

Abhay Zala, Jaemin Cho, Han Lin, Jaehong Yoon, and Mohit Bansal. Envgen: Generating and adapting environments via llms for training embodied agents. *arXiv preprint arXiv:2403.12014*, 2024.

Bofei Zhang, Zirui Shang, Zhi Gao, Wang Zhang, Rui Xie, Xiaojian Ma, Tao Yuan, Xinxiao Wu, Song-Chun Zhu, and Qing Li. Tongui: Internet-scale trajectories from multimodal web tutorials for generalized gui agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 12367–12375, 2026a.

Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–20, 2025.

Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. Infiniteweb: Scalable web environment synthesis for gui agent training. *arXiv preprint arXiv:2601.04126*, 2026b.

Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025.

Table 6: **Supplementary add-only analysis for the supervision-scaling runs.** Adding PhoneWorld supervision on top of the shared AndroidWorld base substantially improves PhoneWorld while leaving AndroidWorld roughly unchanged.

Added PW steps	AndroidWorld	PhoneWorld
0K	46.6	14.2
10K	45.7	64.2
20K	45.2	70.0
36K	46.6	73.3

A Supplementary Add-Only Analysis for Full Replacement

Table 6 reports the AndroidWorld and PhoneWorld results for the add-only supervision-scaling runs discussed in Section 5.3. The shared AndroidWorld base corpus is fixed, and we add increasing amounts of PhoneWorld supervision without removing any existing AndroidWorld data.